

TECH FORCE: GALACTIC
ARCHITECTURE

BINÁRIOS WINDOWS



5

LISTA DE FIGURAS

Figura 1 - Estrutura do arquivo PE	12
Figura 2 – Valores do cabeçalho DOS utilizando o PE-Bear	14
Figura 3 – Início do NT Header	14
Figura 4 – Exemplo do Rich Header do cmd.exe descriptografado	17
Figura 5 – Exemplo de valores do cabeçalho Rich utilizando o PE-Bear	17
Figura 6 - Identificando o cabeçalho PE usando o HxD Editor	19
Figura 7 – Estrutura do File Header utilizando o CFF Explorer.....	19
Figura 8 – Exemplo da estrutura do Cabeçalho Opcional utilizando o CFF Explorer	21
Figura 9 - Exemplo entre 32 e 64 bit.....	25
Figura 10 - Exemplo da estrutura do Data Directory utilizando o PE-Bear	26
Figura 11 - Seções do binário.....	26
Figura 12 – Exemplo do arquivo PE em disco e em memória	29
Figura 13 – Exemplo da função MessageBox gravada na IAT	34
Figura 14 – Exemplo de IAT Hiding.....	36
Figura 15 – Composição de um sistema operacional I	38
Figura 16 – Composição de um sistema operacional II	39

LISTA DE TABELAS

Tabela 1 – Seções do cabeçalho PE.....	27
Tabela 2 – As entradas para as importações de uma determinada DLL	33

EMANIP

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Estrutura do IMAGE_DOS_HEADER	13
Código-fonte 2 – Estrutura 32 e 64 bits do NT Header	18
Código-fonte 3 – Estrutura do Data Directory IMAGE_DATA_DIRECTORY	25
Código-fonte 4 – IMAGE_SECTION_HEADER	28
Código-fonte 5 – Exemplo da estrutura de Bound Import IMAGE_BOUND_IMPORT_DESCRIPTOR	30
Código-fonte 6 – Exemplo da estrutura IMAGE_IMPORT_DESCRIPTOR	31
Código-fonte 7 – Exemplo de uso da IAT com a função MessageBoxA	34
Código-fonte 8 – Exemplo de chamada dinâmica de função (IAT Hiding)	35

SUMÁRIO

BINÁRIOS WINDOWS	6
1 INTRODUÇÃO	6
2 UMA BREVE HISTÓRIA.....	7
3 TIPOS DE BINÁRIOS.....	9
3.1 Arquivos EXE.....	9
3.2 Arquivos DLL	9
3.3 Arquivos LIB	10
3.4 Arquivos SYS.....	11
4 FORMATO PE.....	12
4.1 DOS Header	13
4.1.1 DOS Stub	15
4.2 Rich Header.....	15
4.2.1 Estrutura do Rich Header	16
4.3 NT Header (IMAGE_NT_HEADERS)	18
4.3.1 Signature	19
4.3.2 File Header (IMAGE_FILE_HEADER)	19
4.3.3 Optional HEader (IMAGE_OPTIONAL_HEADER)	20
4.3.4 64-bit version	24
4.3.5 Data Directories	25
4.4 Section & Sections Header	26
4.4.1 SEction Header.....	27
4.5 Bound Imports	30
4.6 Import Direcotory Table (IDT)	31
4.7 Import Lookup Table (ILT)	32
4.8 Import Address Table (IAT)	33
4.8.1 Iat Hiding	34
4.9 Base Reallocation.....	36
5 EXECUTANDO BINÁRIO WINDOWS EM SISTEMAS GNU/LINUX	37
5.1 Vantagens do WINE	39
5.2 Instalando o WINE.....	40
CONCLUSÃO	41
REFERÊNCIAS	42

BINÁRIOS WINDOWS

1 INTRODUÇÃO

O mundo do desenvolvimento de software é repleto de complexidades e desafios. Em meio a essa vastidão, um elemento essencial que permeia a criação e execução de aplicativos no sistema operacional Windows, é o formato do arquivo PE, também conhecido como *Portable Executable*. Neste capítulo, iremos desvendar as profundezas de suas estruturas internas, desvendando sobre os seus detalhes e revelando a importância de seu papel para a execução de programas.

Veremos como o formato PE é a base que permite ao sistema operacional entender e executar binários. Abordaremos os diferentes tipos de binário Windows que utilizam o formato PE como o (.exe, .dll, .sys) entre outros.

Entenderemos como esse formato estabeleceu o padrão para a organização e interação entre os diversos componentes no ecossistema Windows.

Ao longo deste capítulo, também discutiremos técnicas e a utilização de algumas ferramentas, como o PE-Bead, CFF Explorer, PEstudio, (entre outras) que nos permite analisar, manipular e entender a fundo suas estruturas.

No final deste capítulo, você terá uma compreensão sólida do formato PE, suas nuances e sua importância no desenvolvimento e análise de software para ambiente Microsoft Windows. Este conhecimento irá ajudá-lo em diferentes campos da cibersegurança, como (análise de malware, engenharia reversa, exploração e desenvolvimento de exploits), pois permitirá uma boa compreensão de como os aplicativos forma desenvolvidos, permitindo decifras os mistérios por trás de um binário.

Então, prepare-se par uma imersão fascinante no coração do *Portable Executable*, enquanto desventramos suas estruturas internas e desbloqueamos o poder do formato que impulsiona a execução de programas no sistema Windows.

2 UMA BREVE HISTÓRIA

O formato do arquivo *Portable Executable* (também conhecido como PE), possui uma história rica e fascinante, que retornam aos primórdios da computação moderna. Para entendermos sua evolução, é necessário voltarmos no tempo e explorar as necessidades e origem desse formato.

Foi por volta da década de 1980, mais especificamente em 1981 que foi quando a Microsoft lançou a primeira versão do sistema operacional MS-DOS (Microsoft Disk Operating System) essa foi uma evolução dos sistemas DOS utilizados na época. Esse sistema foi desenvolvido para uma linha de computadores pessoais (PC) IBM-PC, este foi concebido para ser um sistema operacional de 16 bit, monoprogramável, monousuário e com uma interface de linha de comando (também conhecido como CLI). Naquela época, as aplicações eram executadas por meio de arquivos que continha a extensão (.com – também conhecido como Command File), esses arquivos eram simplesmente uma sequência de bytes contendo instruções de código de máquina, similar a um *payload* (carga útil). No entanto, esses arquivos não forneciam muitas informações sobre a estrutura interna ou as dependências dos programas, o que acabava dificultando para a criação, a depuração (*debugging*) de aplicações.

No ano de 1983, a Microsoft anunciou sua intenção de desenvolver um sistema que possuía uma GUI (*Graphical User Interface* – interface gráfica de usuário), foi daí que veio o nome para a primeira versão do Windows, mas especificamente o Windows 1.0. Essa nova versão do sistema executava sobre o MS-DOS, e nos introduziu a um novo formato para arquivos executáveis, o *New Executable* (ou NE).

Esse novo formato trouxe algumas melhorias para a época quando se comparado aos arquivos (.com). O formato NE possibilitava a organização dos aplicativos em módulos (ou partições), além de fornecer informações relevantes como os cabeçalhos e as tabelas de importação e exportação de funções utilizadas pelo binário. Essa evolução possibilitou o desenvolvimento de programas mais complexos e facilitando no desenvolvimento de bibliotecas compartilhadas.

Já no ano de 1993, foi lançado a primeira versão do sistema Windows NT (ou Windows New Technology), o Windows NT 3.1. Esse sistema foi desenvolvido baseado no MS-DOS, e nessa versão foi introduzidas as famosas Windows 32 API (também conhecidas como Win32 APIs). Essas APIs (*Application Programming Interfaces*) foram desenvolvidas e distribuídas pela Microsoft para facilitar no desenvolvimento de software e drives compatíveis com o sistema Windows. E facilitar a parte de manutenção do kernel por parte deles.

Na época o formato NE ofi estendido e aprimorado para se tornar o formato PE, esse foi baseado no formato COFF (*Common Object File Format*) que também foi bastante utilizado em sistemas Unix.

O formato PE foi projetado para ser portátil e flexível, permitindo a execução de aplicativos em diferentes ambientes Windows, de forma a se independente da arquitetura do processador, sendo x86 (32-bit) ou x64 (64-bit). Esse formato nos fornece uma forma estruturada e organizada para os arquivos executáveis, incluindo cabeçalhos relevantes que podem no ajudar por exemplo uma análise de malware, ou a engenharia reversa de software proprietários. Essa estrutura permite com que o sistema operacional carregue e execute a aplicação de uma forma eficiente, garantindo sua compatibilidade com a diferentes versões do Windows.

Desde então, o formato PE passou por diversas evoluções e atualizações ao logo dos anos junto ao desenvolvimento e melhoria do Windows. Novos recursos foram adicionados, como o controle de acesso a recursos, suporte a imagens de 64-bit (também conhecida como PE32+), entre outras.

Hoje, o formato PE é amplamente utilizado em todas as versões modernas do Widnows, desde o Windows NT 3.1, Windows XP, até as versões mais recentes como o Windows 10 e o 11. Essa estrutura se tornou a base fundamental para o desenvolvimento de aplicações para o sistema Windows, provendo uma estrutura robusta e versátil para criação programas complexos.

Como podemos ver com uma breve história o aprimoramento e adaptação ao longo das décadas, permitiu com que o formato PE continuasse sendo uma peça-chave no mundo da programação focado em sistemas Windows.

Agora que fizemos uma curta viagem no tempo. Vamos entrar a fundo e entender os diferentes binários do sistema Windows que utilizam esse formato e em seguida entenderemos como eles são compostos.

3 TIPOS DE BINÁRIOS

O sistema operacional Windows possui diferente tipos de binário que fazem o uso do formato PE. Alguns deles são, ACM (Audio Compression Manager), CPL (Control Panel), SRC (Source Code), entre outros como veremos a seguir.

Então agora vamos focar nas extensões de binários que são comumente vistos nos sistemas Windows.

3.1 Arquivos EXE

Iniciaremos como o formato mais conhecido o famoso (.exe). Esse são arquivos que contêm o código do executável de um programa. Os executáveis PE podem ser aplicativos de desktop, ou utilitários de linha de comando (*Command Line Util*), por exemplo, o comando `ping.exe`, esse é um utilitário de rede utilizado para validar se outro host está vivo (live) na rede. Outro exemplo são os instaladores para outros softwares, como o `OfficeSetup.exe`.

3.2 Arquivos DLL

Os arquivos de biblioteca de vínculo dinâmico (DLL - *Dynamcy Link Libraries*) são as que contêm funções e recursos compartilhados que podem ser usados por vários programas. Esse arquivo facilita na reutilização de código e na modularização de programas. Alguns arquivos (.dll) que você encontrar por padrão em qualquer binário são:

- **Ntdll.dll:** A ntdll.dll (Library for NT-based operating System – Biblioteca para Sistemas operacionais baseando em NT) é uma biblioteca essencial para o sistema Windows. Ela contém funções fundamentais e serviços de baixo nível que são utilizados pelo kernel do Windows. Essa DLL fornece uma interface para acessar recursos de hardware, gerenciamento de memória, manipulação de exceções e operações de I/O (ou Input or Output – Entrada ou Saida). Essa é carregada em todos os processos do Windows.
- **Kernel32.dll:** A kernel32.dll é outra biblioteca fundamental para o funcionamento dos binários Windows. Ela fornece funções básicas de gerenciamento de memória, processos, threads, arquivos e I/O. Essa DLL é responsável por várias tarefas essenciais, como inicialização do sistema, carregamento de drives, criação de processos e manipulação de recursos. Ela também serve como uma camada de abstração entre os aplicativos e o kernel do Windows, fornecendo uma interface de programação (ou API) consistente e confiável.
- **User32.dll:** A user32.dll é outra biblioteca do Windows. Ela contém funções relacionadas a interface do usuário (ou UI). Ela oferece suporte à criação e gerenciamento de janelas, controle de eventos do mouse e do teclado, manipulação de gráficos, menus e diálogos.

Essas três DLLs, são peças essenciais do sistema operacional Windows e desempenham papéis cruciais no funcionamento do sistema.

3.3 Arquivos LIB

Embora não sejam executáveis em si, as bibliotecas dinâmicas (DLLs) podem ser acompanhadas por arquivos de importação (.lib). Esses arquivos contêm informações de vínculo usadas pelos compiladores durante o processo de compilação de um programa. Esses arquivos (.lib) são usados para facilitar a vinculação do programa com as DLLs corretas durante o tempo de execução.

3.4 Arquivos SYS

Esses são arquivos que possuem códigos necessários para controlar dispositivos de hardware específicos. Os drives de dispositivos são essenciais para que o sistema operacional possa interagir com componentes físicos do computador, como placa de (vídeo, som, rede), ou impressoras. Alguns exemplos de arquivos (.sys) utilizados pelo sistema Windows.

- **tcpip.sys:** Este arquivo é responsável pela implementação do protocolo TCP/IP (Transmission Control Protocol/Internet Protocol) no Windows. Seu principal objetivo é lidar com a comunicação de rede, incluindo o acesso à internet.
- **usbport.sys:** Este arquivo é responsável pelo gerenciamento de portas USB (Universal Serial Bus). Fornecendo as funcionalidades necessários para conectar e utilizar dispositivos USB.

4 FORMATO PE

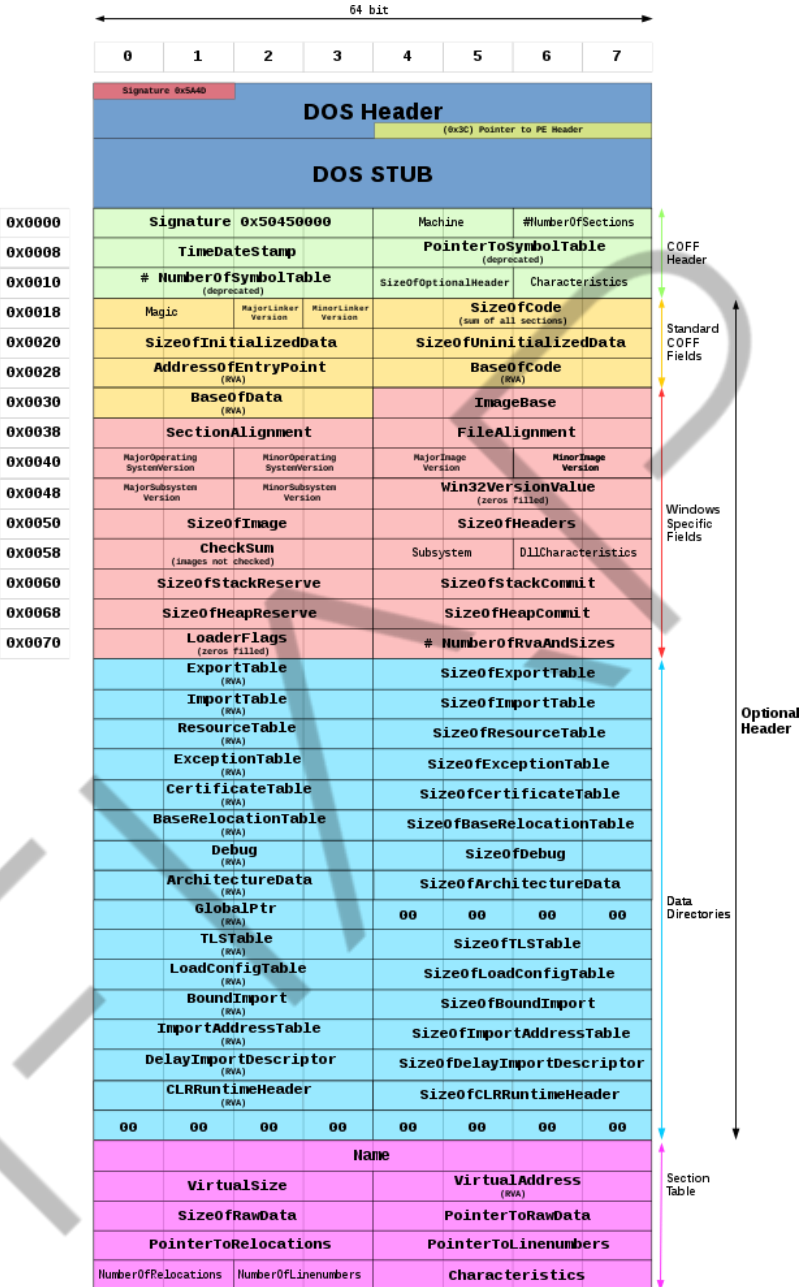


Figura 1 - Estrutura do arquivo PE
Fonte: Google Imagens (2023)

Com base na figura “Estrutura do arquivo PE”, iremos nos guiar para tender a fundo o formato PE. Podemos dizer que o binário Windows é dividido em duas partes, a primeira sendo as estruturas dos Headers, onde são armazenadas diferentes informações sobre o binário, como se ele é 32 ou 64 bits, a quantidade de seções, características do arquivo indicando se ele é um executável (.exe) ou uma biblioteca (.dll).

Outra parte são as Sections (ou Seções), as seções possuem diferentes informações dos diretórios (ou partições) de um executável. Provendo detalhes sobre sua organização e atribuição, veremos mais sobre essas partes em mais detalhes mais a frente.

4.1 DOS Header

O cabeçalho DOS, também conhecido como (MS-DOS Header) é a primeira parte da estrutura de um arquivo executável.

Nas versões mais recentes do Windows essa parte não é tão importante para a funcionalidade dos arquivos PE, porém ela ainda é mantida por questões de compatibilidade com sistemas legados. Sem esse cabeçalho, caso você tente executar um binário em um sistema DOS ele simplesmente não executará, gerando uma mensagem de erro.

Abaixo estão os campos (ou membros) encontrados na estrutura do cabeçalho DOS:

```
typedef struct _IMAGE_DOS_HEADER { // DOS .EXE header
    WORD e_magic; // Magic number
    WORD e_cblp; // Bytes on last page of file
    WORD e_cp; // flages in file
    WORD e_crlc; // Relocations
    WORD e_cparhdr; // Size of header in paragraphs
    WORD e_minalloc; // Minimum extra paragraphs needed
    WORD e_maxalloc; // Maximum extra paragraphs needed
    WORD e_ss; // Initial (relative) SS value
    WORD e_sp; // Initial Sfl value
    WORD e_csum; // Checksum
    WORD e_ip; // Initial lfl value
    WORD e_cs; // Initial (relative) CS value
    WORD e_lfarlc; // File address of relocation table
    WORD e_ovno; // Overlay number
    WORD e_res[4]; // Reserved words
    WORD e_oemid; // OEM identifier (for e_oeminfo)
    WORD e_oeminfo; // OEM information; e_oemid specific
    WORD e_res2[10]; // Reserved words
    LONG e_lfanew; // File address of new exe header
} IMAGE_DOS_HEADER, * fIMAGE_DOS_HEADER;
```

Código-fonte 1 – Estrutura do IMAGE_DOS_HEADER

Fonte: Elaborado pelo autor (2023)

Porém, apenas duas dessas são interessantes para nós, sendo elas:

- **Magic Number:** Esse campo é o `e_magic`, ele possui um tamanho de 2 bytes que identifica o arquivo como um executável do MS-DOS. Esse valor normalmente é representando pela sequência de bytes **0x4D5A**, que

representadas em ASCII significa **MZ**. Uma curiosidade sobre os caracteres MZ, se dá por conta do nome de um dos desenvolvedores do MS-DOS, o *Mark Zbikowski*.

- **Início do NT Header:** Esse é o último membro da estrutura `IMAGE_DOS_HEADER`, o campo `e_lfanew` possui um tamanho de 4 bytes, localizado no offset **0x3C**. Esse é um membro muito importante para o carregamento do PE no sistema Windows.

Offset	Name	Value
0	Magic number	5A4D
2	Bytes on last page of file	90
4	Pages in file	3
6	Relocations	0
8	Size of header in paragraphs	4
A	Minimum extra paragraphs needed	0
C	Maximum extra paragraphs needed	FFFF
E	Initial (relative) SS value	0
10	Initial SP value	B8
12	Checksum	0
14	Initial IP value	0
16	Initial (relative) CS value	0
18	File address of relocation table	40
1A	Overlay number	0
1C	Reserved words[4]	0, 0, 0, 0
24	OEM identifier (for OEM information)	0
26	OEM information; OEM identifier specific	0
28	Reserved words[10]	0, 0, 0, 0, 0, 0, 0, 0, 0, 0
3C	File address of new exe header	100

Figura 2 – Valores do cabeçalho DOS utilizando o PE-Bear
Fonte: Google Imagens (2023)

Como podemos ver na Figura “Valores do cabeçalho DOS utilizando o PE-Bear”, utilizamos uma ferramenta chamada PE-Bear, essa ferramenta foi desenvolvida pela Hasherezade. Vemos que os diferentes valores atribuídos para cada membro, o último membro (`e_lfanew`) no offset 0x3C armazena o valor **0100** que no indica o offset onde inicia o cabeçalho NT (ou NT Header) do nosso binário, com forme mostra na Figura “Início do NT Header”:

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	
100	50	45	00	00	4C	01	05	00	D8	89	A6	64	00	00	00	00		P	E	.	.	L	.	.	.	.	0	.	.		d	.	.	.
110	00	00	00	00	E0	00	02	01	0B	01	0E	23	00	0E	00	00		.	.	.	.	à	.	.	.	.	.	.	.	#	.	.	.	

Figura 3 – Início do NT Header
Fonte: Google Imagens (2023)

Nesse offset já conseguimos ver a assinatura do cabeçalho NT header (ou PE Header).

4.1.1 DOS Stub

De forma bem simples, essa parte do cabeçalho é um pequeno shellcode que tem como objetivo exibir uma mensagem “*This program cannot be run in DOS mode*” (Esse programa não pode ser executado em modo DOS) e simplesmente terminar sua execução. Essa mensagem pode ser alterada pelo desenvolvedor durante o processo de compilação do programa.

4.2 Rich Header

Quando uma aplicação é desenvolvida utilizando o Toolkit do Visual Studio, se é criado o cabeçalho Rich.

Aqui vale ressaltar que esse cabeçalho não faz parte da estrutura do cabeçalho PE. Aplicações que fazem o uso de outros compiladores e linkers não possuem esse cabeçalho.

O Rich Header foi introduzido junto ao Visual Studio 97 SP3, lançado no ano de 1997. Naquela época a Microsoft não anunciou a implementação de uma nova estrutura nem mesmo publicou uma documentação ao seu respeito. Essa até hoje é uma estrutura não documentada oficialmente pela Microsoft. Porém, alguns pesquisadores realizaram a Engenharia Reversa dessa estrutura para descobrir as suas utilidades, dessa forma oferecendo a comunidade um melhor entendimento sobre a tal.

Esse cabeçalho possui dados contendo informações sobre o ambiente de desenvolvimento do projeto.

Até hoje não se sabe exatamente o porquê a Microsoft decidiu adicionar esse cabeçalho nos programas desenvolvidos em sua IDE.

Do ponto de vista de um profissional de segurança, esses dados podem ser de extrema utilidade! Como? Você deve estar se perguntando, para responder essa dúvida vamos pensar com dois pontos de vista, um sendo o atacante e o outro sendo o defensor.

1. **Visão do Atacante:** Como atacante, saber utilizar esse cabeçalho ao seu favor pode aumentar suas chances de sucesso e OpSec (*Operation Security*) em suas campanhas de Red Team. No ano de 2018 ocorreu um ataque contra a Olimpíada de inverno que aconteceu na Coreia do Sul. Esse ataque envolveu um malware que ficou conhecido como Olympic Destroyer. Esse malware foi desenvolvido por um grupo APT que tinha como objetivo atrapalhar a Olimpíada de inverno. O que mais chamou a atenção dos pesquisadores foi que as informações atribuídas no Rich header do binário foram intencionalmente alteradas para atribuir o ataque a outro grupo APT da Coreia do Norte conhecido como Lazarus Group.
2. **Visão do defensor:** Como um analista de malware podemos, por exemplo, utilizar tais informações para obtermos e criarmos um certo entendimento de nossos adversários. Permitindo saber um pouco sobre se “poder de guerra”. Essas informações podem ser de extrema importância para criações de modelos de inteligência artificial que correlacionam diferentes tipos de binários identificando padrões podendo assim correlacionar tais famílias de malware para determinadas campanhas ou atores.

As possibilidades aqui são infinitas tudo depende da sua criatividade.

4.2.1 Estrutura do Rich Header

Como vimos acima, esse cabeçalho possui informações sobre o ambiente de desenvolvimento do programa, será que é por isso que esse cabeçalho é tão rico? Bom, vamos ver...

O Rich Header consiste em um bloco de dados criptografado utilizando um simples cifra de XOR seguida por sua assinatura Rich mais um valor de 32 bit (conhecido como *checksum*) considerado a chave para descriptografar os dados do cabeçalho.

Ao descriptografar os dados desse cabeçalho encontraremos uma sequência de caracteres (**DanS**) que é usado para classificar o início do cabeçalho Rich. Essas assinaturas que determinam o início e o fim do cabeçalho Rich se dá por conta dos desenvolvedores do Visual C++ Linker, Dan Spalding e Richard Shupak.

As informações contidas nesse cabeçalho nos possibilitam entender a natureza e escala de um projeto. Simplesmente olhando para os nomes do ProdID, nesse campo vemos diferentes informações do que foi utilizado na construção do projeto, se o projeto foi escrito em C, C++, Assembly ou Visual Basic. As vezes conseguimos até mensurar quão grande esse projeto é, simplesmente se baseando no product count.

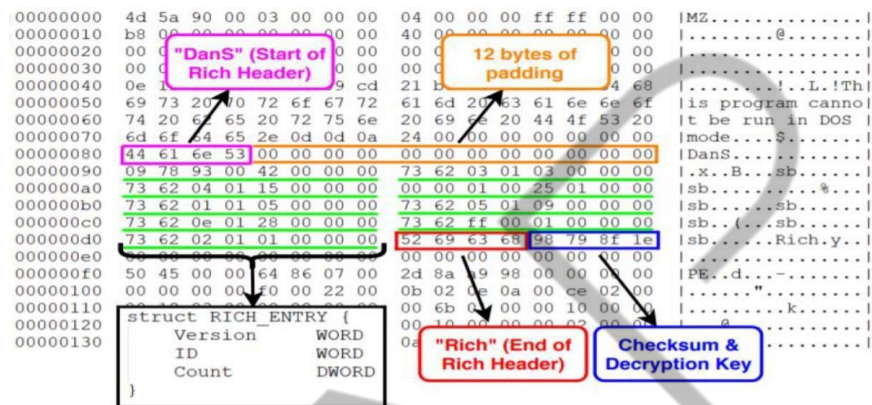


Figura 4 – Exemplo do Rich Header do cmd.exe descriptografado
Fonte: DUBYK (2019)

Como podemos ver na Figura 4, temos um exemplo do cabeçalho Rich descriptografado, os primeiros bytes 0x44616E53 representados em ASCII é a assinatura **DanS**, como dito antes indicando o início do nosso cabeçalho Rich. Um pouco mais abaixo vemos o final com a sequência de bytes **0x52696368** representando nossa assinatura **Rich** e indicando o final do cabeçalho.

O corpo do cabeçalho Rich é composto por uma única matriz que armazena metadados em cada etapa da cadeia de construção durante a vinculação de objetos em um único executável. A matriz é variável em comprimento e composta por uma estrutura de 8 bytes. A estrutura contém a identificação do produto (pID), a versão do produto (pV) e uma contagem (pC) de cada vez que o produto foi usado; o número total de arquivos fonte compilados (DUBYK, 2019, p. 6, tradução nossa).

Offset	Name	Value	Unmasked Value	Meaning	ProductId	BuildId	Count	VS-version
80	DanS ID	4b4f6553	536e6144	DanS				
84	Checksummed padding	18210417	0	0				
88	Checksummed padding	18210417	0	0				
8C	Checksummed padding	18210417	0	0				
90	Comp ID	1821041d18b27c1e	a00937809	30729.147.10	Implib900	30729	10	Visual Studio 2008 09.00
98	Comp ID	18210414192078a8	301017cbf	31935.257.3	Implib1400	31935	3	Visual Studio 2015 14.00
A0	Comp ID	18210404192478a8	1301057cbf	31935.261.19	Utc1900_CPP	31935	19	Visual Studio 2015 14.00
A8	Comp ID	1821041b192578a8	c01047cbf	31935.260.12	Utc1900_C	31935	12	Visual Studio 2015 14.00
B0	Comp ID	18210416192278a8	101037cbf	31935.259.1	Masm1400	31935	1	Visual Studio 2015 14.00
B8	Comp ID	1821041519207c5c	20101784b	30795.257.2	Implib1400	30795	2	Visual Studio 2015 14.00
C0	Comp ID	1821042718200417	3000010000	0.1.48	Import0	0	48	Visual Studio
C8	Comp ID	18210416192979ce	101087dd9	32217.264.1	Utc1900_LTCG_C	32217	1	Visual Studio 2017 14.01+
D0	Comp ID	1821041618de79ce	100ff7dd9	32217.255.1	Cvtrse1400	32217	1	Visual Studio 2015 14.00
D8	Comp ID	18210416192379ce	101027dd9	32217.258.1	Linker1400	32217	1	Visual Studio 2015 14.00
E0	Rich ID	68636952		Rich				
E4	Checksum	18210417	18210417					

Figura 5 – Exemplo de valores do cabeçalho Rich utilizando o PE-Bear
Fonte: Google Imagens (2023)

4.3 NT Header (IMAGE_NT_HEADERS)

O NT Header (também conhecido como PE Header) é uma parte essencial e de extrema importância para os arquivos executáveis do Windows. Esse cabeçalho é composto por várias estruturas de dados que contém informações de como ele deve ser carregado e executado pelo sistema operacional, mas especificamente o *Image Loader* (carregador de imagem). Vamos entender de forma básica sobre o *Image Loader*, visto que ele é algo bastante complexo.

O *Image Loader* é um componente do sistema operacional responsável pelo carregamento e execução de arquivos de imagem, como DLL e executáveis (.exe). Ele desempenha um papel fundamental na execução de programas e no gerenciamento de suas dependências.

Quando um programa é inicializado no Windows, o *Image Loader* é ativado para carregar todas as bibliotecas e arquivos executáveis necessários para executar o programa corretamente. Ele verifica e resolve as dependências do programa, garantindo que todas as DLLs necessário estejam presentes.

Em resumo, o *Image Loader* é responsável por carregar e executar arquivos de imagens (ou executáveis) garantindo que todas as dependências estejam satisfeitas e contribuindo para execução correta e segura dos programas.

Agora vamos entender a fundo a estrutura do NT header. Sua estrutura é definida no arquivo de header `winnt.h`, como `IMAGE_NT_HEADERS`, abaixo é possível ver detalhadamente como essa estrutura é definida.

```
typedef struct _IMAGE_NT_HEADERS64 {  
    DWORD Signature;  
    IMAGE_FILE_HEADER FileHeader;  
    IMAGE_OPTIONAL_HEADER64 OptionalHeader;  
} IMAGE_NT_HEADERS64, * PIMAGE_NT_HEADERS64;
```

```
typedef struct _IMAGE_NT_HEADERS {  
    DWORD Signature;  
    IMAGE_FILE_HEADER FileHeader;  
    IMAGE_OPTIONAL_HEADER32 OptionalHeader;  
} IMAGE_NT_HEADERS32, * PIMAGE_NT_HEADERS32;
```

Código-fonte 2 – Estrutura 32 e 64 bits do NT Header
Fonte: Elaborado pelo autor (2023)

Como podemos ver essa estrutura possui duas versões uma para 32-bit e outra pra 64-bit contendo três membros (ou elementos), o primeiro membro representa uma assinatura, e as duas outras estruturas são `IMAGE_FILE_HEADER` e `IMAGE_OPTIONAL_HEADER`.

4.3.1 Signature

O primeiro membro da estrutura do NT Header é a assinatura PE, esse membro ocupa um valor de **4 bytes** ou `DWORD`, sempre possuindo um valor fixo de **0x5045000** que convertido para ASCII representa **PE\0\0**. Como podemos ver na figura “Identificando o cabeçalho PE usando o HxD Editor”.

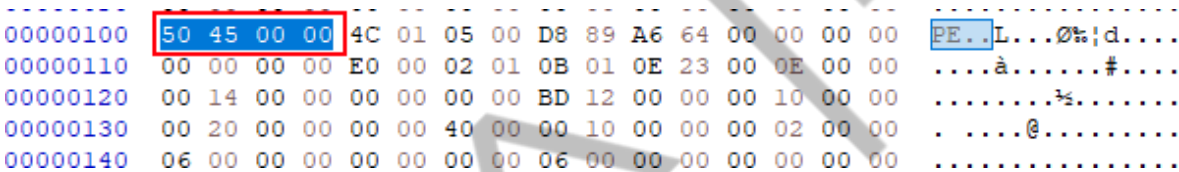


Figura 6 - Identificando o cabeçalho PE usando o HxD Editor
Fonte: Google Imagens (2023)

4.3.2 File Header (`IMAGE_FILE_HEADER`)

Também conhecido como COFF Header, essa estrutura armazena diferentes informações sobre o arquivo PE.

Member	Offset	Size	Value	Meaning
Machine	00000104	Word	014C	Intel 386
NumberOfSections	00000106	Word	0005	
TimeDateStamp	00000108	Dword	64A689D8	
PointerToSymbolTable	0000010C	Dword	00000000	
NumberOfSymbols	00000110	Dword	00000000	
SizeOfOptionalHeader	00000114	Word	00E0	
Characteristics	00000116	Word	0102	Click here

Figura 7 – Estrutura do File Header utilizando o CFF Explorer
Fonte: Google Imagens (2023)

Utilizando o CFF Exploerer para analisarmos a estrutura do File Header. Vemos que ele possui 8 membros sendo eles:

- **Machine:** Esse membro possui um tamanho de **2 bytes**, ele armazena o tipo de arquitetura de CPU para qual o programa foi desenvolvido. Na documentação oficial da Microsoft possui uma tabela de diferentes valores que se pode haver nesse campo. Os dois mais comuns que você irá encontrar será o **0x014C** que representa a arquitetura **i386** e **0x8664** para **AMD64**.
- **NumberOfSections:** Esse membro possui um tamanho de **2 bytes**, como o próprio nome sugere, é armazenado o número de seções que o executável possui.
- **TimeStamp:** Esse é um membro de **4 bytes**, que armazena a data e o horário em que o binário foi criado, o formato utilizado é do sistema Unix.
- **PointerToSymbolTable e NumberOfSymbols:** Cada um desse elemento possuem um tamanho de **4 bytes**. Eles armazenam o offset do arquivo para a tabela de símbolos COFF e o número de entrada dessa tabela, no entanto, são definidos como zero, o que significa que nenhuma tabela COFF está presente.
- **SizeOfOptionalHeader:** Esse membro armazena o tamanho (de **2 bytes**) do Optional Header.
- **Characteristics:** Esse membro possui um tamanho de **2 bytes**, contendo os atributos do arquivo, que indicam se ele é um executável ou uma biblioteca dinâmica. Neste nosso exemplo temos o valor **0x0102** representando um binário de 32-bit, como podemos ver nas constantes abaixo:
 - **IMAGE_FILE_32BIT_MACHINE (0x0100)** – Versão para arquitetura de 32-bit.
 - **IMAGE_FILE_EXECUTABLE_IMAGE (0x0002)** – Indica que a imagem é válida e pode ser executada.

4.3.3 Optional Header (IMAGE_OPTIONAL_HEADER)

Essa é a parte mais importante do cabeçalho NT, aqui o carregador de PE busca por informações específicas que são necessárias para carregar e executar corretamente o binário.

Como o nome sugere, alguns dos membros dessa estrutura são opcionais podendo ser especificada pelo desenvolvedor. Esse cabeçalho não possui um

tamanho fixo, pois isso a existência e o uso do membro **IMAGE_FILE_HEADER_SizeOfOptionalHeader**. Arquivos Objetos (ou Object Files), que veremos mais a frente, não possuem esse header.

Member	Offset	Size	Value	Meaning
Magic	00000118	Word	010B	PE32
MajorLinkerVersion	0000011A	Byte	0E	
MinorLinkerVersion	0000011B	Byte	23	
SizeOfCode	0000011C	Dword	00000E00	
SizeOfInitializedData	00000120	Dword	00001400	
SizeOfUninitializedData	00000124	Dword	00000000	
AddressOfEntryPoint	00000128	Dword	000012BD	.text
BaseOfCode	0000012C	Dword	00001000	
BaseOfData	00000130	Dword	00002000	
ImageBase	00000134	Dword	00400000	
SectionAlignment	00000138	Dword	00001000	
FileAlignment	0000013C	Dword	00000200	
MajorOperatingSystemVersion	00000140	Word	0006	
MinorOperatingSystemVersion	00000142	Word	0000	
MajorImageVersion	00000144	Word	0000	
MinorImageVersion	00000146	Word	0000	
MajorSubsystemVersion	00000148	Word	0006	
MinorSubsystemVersion	0000014A	Word	0000	
Win32VersionValue	0000014C	Dword	00000000	
SizeOfImage	00000150	Dword	00006000	
SizeOfHeaders	00000154	Dword	00000400	
Checksum	00000158	Dword	00000000	
Subsystem	0000015C	Word	0003	Windows Console
DllCharacteristics	0000015E	Word	8140	Click here
SizeOfStackReserve	00000160	Dword	00100000	
SizeOfStackCommit	00000164	Dword	00001000	
SizeOfHeapReserve	00000168	Dword	00100000	
SizeOfHeapCommit	0000016C	Dword	00001000	
LoaderFlags	00000170	Dword	00000000	
NumberOfRvaAndSizes	00000174	Dword	00000010	

Figura 8 – Exemplo da estrutura do Cabeçalho Opcional utilizando o CFF Explorer
Fonte: Google Imagens (2023)

- **Magic:** Esse membro contém um valor do tipo inteiro que identifica o estado da imagem. Os dois valores mais comuns são:
 - **0x10B** – Identifica a imagem com um binário de 32 bit (ou PE32).
 - **0x20B** – Identifica a imagem como binário de 64 bit (ou PE32+).
- **MajorLinkerVersion e MinorLinkerVersion:** Esses são membros do tamanho de **1 byte**, que armazenam a maior e a menor versão do linker.
- **SizeOfCode:** Esse membro possui o tamanho de **4 bytes**, armazenando o tamanho do código da seção (**.text**), ou a soma de todo o código.
- **SizeOfInitializedData:** Esse membro de **4 bytes** armazena o tamanho dos dados inicializáveis na seção (**.data**), ou a soma de todos os dados inicializáveis caso haja múltiplas sessões.

- **SizeOfUninitializedData:** Esse membro de **4 bytes** armazena o tamanho dos dados que não são inicializáveis na seção (**.bss**), ou a soma de todos os dados não inicializáveis caso haja múltiplas sessões.
- **AddressOfEntryPoint:** Esse membro do tamanho de **4 Bytes** contém um RVA (*Relative Virtual Address*) que representa o **Entry Point de quando o arquivo é carregado para a memória primária**. Para imagens de programa, este é o endereço inicial. Para drivers de dispositivo, este é o endereço da função de inicialização. Um ponto de entrada é opcional para DLLs. Quando nenhum ponto de entrada está presente, este campo deve ser zero.
- **BaseOfCode (4 Bytes):** Esse membro armazena o RVA para o início da seção de código quando um binário é carregado para a memória.
- **BaseOfData (apenas 32-bit – 4 Bytes):** Contém um RVE do início da seção que contém os dados necessário para o arquivo ser carregado para a memória.
- **ImageBase (4 Bytes):** Esse campo armazena o valor do endereço preferido do primeiro byte da imagem quando é carregada para memória (também conhecido como Endereços Base Preferido), esse valor deve ser um múltiplo de 64K. Devido a proteções de memória como ASLR, o endereço especificado nesse campo quase nunca é utilizado, nesse caso o PE Loader escolher um espaço de memória que não esteja em uso e carrega o binário, depois de ser carregar o binário à memória o Loader realiza um processo de realocação. Há uma seção especial que contém informações sobre locais que precisarão de conserto se a realocação for necessária, essa seção é chamada de seção realocação (**.reloc**), que será falado mais a frente.
- **SectionAlignment:** Esse membro de **4 bytes** contém o alinhamento necessário (em bytes) das seções quando são carregadas na memória. Deve ser maior ou igual a *FileAlignment*. O padrão é o tamanho da página para a arquitetura.
- **FileAlignment:** Similar ao SectionAlignment, possui **4 Bytes** e campo contém um valor que é usado para o alinhamento de *raw data* (ou dados brutos) da seção no disco (em bytes). Permitindo assim manter os limites de alinhamento. A documentação afirma que esse valor deve ser uma potência de 2 entre 512 e 64K e, se o valor de SectionAlignment for menor que o tamanho da página

da arquitetura, os tamanhos de *FileAlignment* e *SectionAlignment* devem corresponder.

- **MajorOperatingSystemVersion e MinorOperatingSystemVersion (2 Bytes):** O maior e o menor número da versão do sistema operacional necessário.
- **MajorImageVersion e MinorImageVersion (2 Bytes):** O maior e o menor número da versão da imagem.
- **MajorSubsystemVersion e MinorSubsystemVersion (2 Bytes):** O maior e o menor número da versão principal do subsistema.
- **Win32VersionValue (4 Bytes):** De acordo com a documentação da Microsoft, esse é um membro Reservado, tenho como valor padrão 0.
- **SizeOfImage (4 Bytes):** O tamanho (em bytes) da imagem, incluindo todo os cabeçalhos, conforme a imagem é carregada na memória. Deve ser um múltiplo de *SectionAlignment*.
- **SizeOfHeaders (4 Bytes):** O tamanho combinado de um MS-DOS stub, cabeçalho PE e cabeçalhos de seção arredondado para um múltiplo de *FileAlignment*.
- **Checksum (4 Bytes):** A soma de verificação do arquivo de imagem. O algoritmo para calcular a soma de verificação é incorporado ao **IMAGHELP.DLL**. Os itens a seguir são verificados para validação no momento do carregamento: todos os drivers, qualquer DLL carregada no momento da inicialização e qualquer DLL carregada em um processo crítico do Windows.
- **Subsystem (2 Bytes):** O subsistema necessário para executar esta imagem.
- **DllCharacteristics (2 Bytes):** Esse campo define algumas características do arquivo executável, como por exemplo, compatibilidade NX e se o programa pode ser realocado em tempo de execução. Quando você está analisando um malware por exemplo, é neste campo que você pode desabilitar a funcionalidade de ASLR, assim facilitando sua análise.
- **SizeOfStackReserve, SizeOfStackCommit, SizeOfHeapReserve, SizeOfHeapCommit (4 Bytes):** Esses campos especificam o tamanho da pilha a ser reservada, o tamanho da pilha a ser confirmada, o tamanho do espaço de heap local a ser reservado e o tamanho do espaço de heap local a ser confirmado, respectivamente.

- **LoaderFlags (4 Bytes):** De acordo com a documentação da Microsoft, esse é um membro Reservado, tenho como valor padrão 0.
- **NumberOfRvaAndSizes (4 Bytes):** O número de entradas do diretório de dados no restante do cabeçalho opcional. Cada um descreve um local e tamanho.

Existe mais um membro que faz parte da estrutura do Optional Header, porém iremos vê-lo logo a seguir.

4.3.4 64-bit version

Como mencionado anteriormente, existem duas versões do Optional Header, uma para 32-bit e outra para 64-bit. As diferenças entre elas são:

- **O tamanho da própria estrutura** (ou o número de membros definidos dentro da estrutura): `IMAGE_OPTIONAL_HEADER32` possui **31 membros** enquanto `IMAGE_OPTIONAL_HEADER64` possui apenas **30 membros**, esse membro adicional na versão de 32-bits é um `DWORD` chamado `BaseOfData` que contém um RVA do início da seção de dados.
- **O tipo de dados de alguns membros:** Os 5 membros a seguir da estrutura de cabeçalho opcional são definidos como `DWORD` na versão de 32 bits e como `ULONGLONG` (ou `QWORD`) na versão de 64 bits:
 - `ImageBase`.
 - `SizeOfStackReserve`.
 - `SizeOfStackCommit`.
 - `SizeofHeapReserve`.
 - `SizeOfHeapCommit`.

ImageBase	00000134	Dword	00400000	32-bit
SizeOfStackReserve	00000160	Dword	00100000	
SizeOfStackCommit	00000164	Dword	00001000	
SizeOfHeapReserve	00000168	Dword	00100000	
SizeOfHeapCommit	0000016C	Dword	00001000	

ImageBase	00000120	Qword	0000000140000000	64-bit
SizeOfStackReserve	00000150	Qword	0000000000100000	
SizeOfStackCommit	00000158	Qword	0000000000001000	
SizeOfHeapReserve	00000160	Qword	0000000000100000	
SizeOfHeapCommit	00000168	Qword	0000000000001000	

Figura 9 - Exemplo entre 32 e 64 bit
Fonte: Google Imagens (2023)

4.3.5 Data Directories

O Diretório de Dados (ou Data Directories) é o último membro da estrutura `IMAGE_OPTIONAL_HEADER`, esse é outra estrutura `IMAGE_DATA_DIRECTORY` que consiste em uma série de entradas, cada uma representando um diretório de dados específico. Cada entrada é uma estrutura de dados de 8 bytes que contém duas informações principais:

1. O endereço virtual do diretório de dados.
2. E o tamanho (em bytes) desses diretórios.

Abaixo podemos ver a representação dessa estrutura:

```
typedef struct _IMAGE_DATA_DIRECTORY {
    DWORD VirtualAddress;
    DWORD Size;
} IMAGE_DATA_DIRECTORY, * PIMAGE_DATA_DIRECTORY;
```

Código-fonte 3 – Estrutura do Data Directory `IMAGE_DATA_DIRECTORY`
Fonte: Elaborado pelo autor (2023)

Então basicamente, essa estrutura armazena dados localizados em cada seção (que veremos logo a seguir) do arquivo PE. Essas informações são de extrema importância para carregar o executável para a memória. Um exemplo de diretório importante é o Import Directory, essa contém informações sobre as funções que foram importadas pelo programa. Veremos mais sobre ela logo adiante.

Existem técnicas que permite um atacante “esconder” algumas funções do diretório as chamando em tempo de execução. Essa técnica é conhecida como *IAT Hidding*.

Abaixo podemos ver essa estrutura e seus valores utilizando o PE-Bear:

	Data Directory	Address	Size
178	Export Directory	0	0
180	Import Directory	25FC	A0
188	Resource Directory	4000	1E0
190	Exception Directory	0	0
198	Security Directory	0	0
1A0	Base Relocation Table	5000	15C
1A8	Debug Directory	21F8	70
1B0	Architecture Specific Data	0	0
1B8	RVA of GlobalPtr	0	0
1C0	TLS Directory	0	0
1C8	Load Configuration Directory	2138	40
1D0	Bound Import Directory	0	0
1D8	Import Address Table	2000	C4
1E0	Delay Load Import Descriptors	0	0
1E8	.NET header	0	0

Figura 10 - Exemplo da estrutura do Data Directory utilizando o PE-Bear
Fonte: Elaborado pelo autor (2023)

Caso um valor seja zero, significa que o diretório de dados não está em uso.

4.4 Section & Sections Header

As seções de cabeçalhos PE contêm os dados do arquivo executável de fato, basicamente esse é a nossa segunda do binário Windows., ocupando o resto do arquivo depois do cabeçalho PE.

Cada uma dessas seções possui um nome específico e seu propósito, vamos ver cada uma dessa e entender melhor sobre o seu uso.

Name	Raw Addr.	Raw size	Virtual Addr.	Virtual Size	Characteristics	Ptr to Reloc.	Num. of Reloc.	Num. of Linenum.
▼ .text	400	E00	1000	CF4	60000020	0	0	0
>	1200	^	2000	mapped: 1000	r-x			
▼ .rdata	1200	C00	2000	B86	40000040	0	0	0
>	1E00	^	3000	mapped: 1000	r--			
▼ .data	1E00	200	3000	388	C0000040	0	0	0
>	2000	^	4000	mapped: 1000	rw-			
▼ .rsrc	2000	200	4000	1E0	40000040	0	0	0
>	2200	^	5000	mapped: 1000	r--			
▼ .reloc	2200	200	5000	15C	42000040	0	0	0
>	2400	^	6000	mapped: 1000	r--			

Figura 11 - Seções do binário
Fonte: Google Imagens (2023)

Nome da Seção	Descrição
.text	Essa seção contém o código a ser executado pelo programa. Por padrão são atribuídas as permissões de <i>leitura e execução</i> . Com as seguintes flags: • IMAGE_SCN_MEM_READ (0x40000000) – A seção pode ser lida. • IMAGE_SCN_MEM_EXECUTE (0x20000000) – A seção pode ser executada. • IMAGE_SCN_CNT_CODE (0x00000020) – A seção contém código executável.
.data	Essa seção contém dados que são inicializáveis pelo programa. Por padrão são atribuídas as permissões de <i>leitura e escrita</i> . Com as seguintes flags: • IMAGE_SCN_CNT_INITIALIZED_DATA (0x00000040) – A seção contém dados de inicialização. • IMAGE_SCN_MEM_READ (0x40000000) – A seção pode ser lida. • IMAGE_SCN_MEM_WRITE (0x80000000) – A seção pode ser escrita.
.rdata	Essa seção contém dados que só podem ser lidos. Por padrão apenas se é atribuída a permissão de <i>leitura</i> . Contendo as flags: • IMAGE_SCN_MEM_READ (0x40000000) – A seção pode ser lida. • IMAGE_SCN_CNT_INITIALIZED_DATA (0x00000040) – A seção contém dados de inicialização.
.edata	Essa seção contém a Tabela de Exportação.
.idata	Essa seção contém a Tabela de Importação.
.pdata	Essa seção contém a Endereços da Tabela de Exceção.
.reloc	Essa seção contém informações sobre a Tabela de Endereços que precisam ser realocados.
.rsrc	Contém recursos necessário para o programa, como por exemplo: imagens, icons. Essa seção é comumente utilizada por atores maliciosos para armazenar <i>payloads</i> que serão executados no sistema.
.tls	Thread Local Storage (ou TLS), essa seção armazena código de threads do programa. Essa seção é comumente utilizada por malware para detectar se estão sendo executados no contexto de um depurador (<i>debugger</i>).
.debug	Essa seção é usada em arquivos de objeto para conter informações de <i>debugger</i> geradas pelo compilador e em arquivos de imagem para conter todas as informações de depuração geradas.

Tabela 1 – Seções do cabeçalho PE
Fonte: Google Imagens (2023)

Essa tabela irá te ajudar com um overview sobre cada uma das seções comumente vista em um arquivo PE.

4.4.1 SEction Header

Logo após o Optional Header e antes das seções encontramos o cabeçalho de Seção (ou Sections Headers). Esse cabeçalho contém informações das seções descritas assim, que um arquivo PE pode vir a ter. Sua estrutura é definida no arquivo `winnt.h` como `IMAGE_SECTION_HEADER`, contendo os seguintes membros.

```
typedef struct _IMAGE_SECTION_HEADER {  
    BYTE Name[IMAGE_SIZEOF_SHORT_NAME];  
    union {  
        DWORD PhysicalAddress;  
        DWORD VirtualSize;  
    } Misc;  
    DWORD VirtualAddress;  
    DWORD SizeOfRawData;  
    DWORD PointerToRawData;  
    DWORD PointerToRelocations;  
    DWORD PointerToLinenumbers;  
    WORD NumberOfRelocations;  
    WORD NumberOfLinenumbers;  
    DWORD Characteristics;  
} IMAGE_SECTION_HEADER, * PIMAGE_SECTION_HEADER;
```

Código-fonte 4 – IMAGE_SECTION_HEADER
Fonte: Elaborado pelo autor (2023)

Então vamos dar uma olhada em cada um desses membros para podermos entender melhor sua usabilidade.

- **Name:** Esse campo armazena uma lista de bytes do tamanho do `IMAGE_SIZEOF_SHORT_NAME` que possui o valor de **8** significando que o nome da seção só pode conter 8 caracteres.
- **PhysicalAddress ou VirtualSize:** Esses campos são definidos como uma **union**, o tipo `union` na linguagem C/C++ são dados definidos pelo desenvolvedor, todos os membros da `union` compartilham a mesma localização de memória. Essa definição significa que, a qualquer momento, uma união não pode conter mais de um objeto de sua lista de membros. Isso também significa que não importa quantos membros uma união tenha, **ela sempre usa apenas memória suficiente para armazenar o maior membro**.
- **VirtualAddress:** Esse membro armazena os primeiros bytes do endereço das seções relativas ao image base quando o binário é carregado para memória.
- **SizeOfRawData:** Esse campo armazena o tamanho da seção no disco. De acordo com a documentação da Microsoft, esse membro deve ser um múltiplo do `IMAGE_OPTIONAL_HEADER.FileAlignment`.
- **PointerToRawData:** Um ponteiro para a primeira página da seção do arquivo.
- **PointerToRelocations:** Um ponteiro de arquivo para o início das entradas de realocação para a seção. É definido como 0 para arquivos executáveis.

- **PointerToLineNumbers:** O ponteiro de arquivo para o início das entradas de número de linha para a seção. Isso é definido como zero se não houver números de linha COFF. Esse valor deve ser zero para uma imagem porque as informações de depuração COFF são obsoletas.
- **NumberOfRelocations:** O número de entradas de realocação para a seção é definido como zero para imagens executáveis.
- **NumberOfLinenumbers:** O número de entradas de realocação para a seção. Esse valor deve ser definido como zero para uma imagem porque as informações de depuração do COFF são obsoletas.
- **Characteristics:** As flags que descreve a características da seção, como visto anteriormente.

Os membros *SizeOfRawData* e *VirtualSize* as vezes podem ser diferentes, isso ocorre por diferentes motivos. Como mencionado acima, o valor do *SizeOfRawData* dever ser um múltiplo do *FileAlignment* um membro do `IMAGE_OPTIONAL_HEADER`, como visto anteriormente.

Por exemplo, caso o *VirtualSize* for menor, o restante da seção será preenchido com zeros. Como o campo *SizeOfRawData* é arredondado, mas o campo de *VirtualSize* não, é possível que o *SizeOfRawData* seja maior que o *VirtualSize* também. Porém quando uma seção é carregada para a memória ela não segue o alinhamento dessa forma simplesmente alinhado o tamanho necessário que a seção irá ocupar.

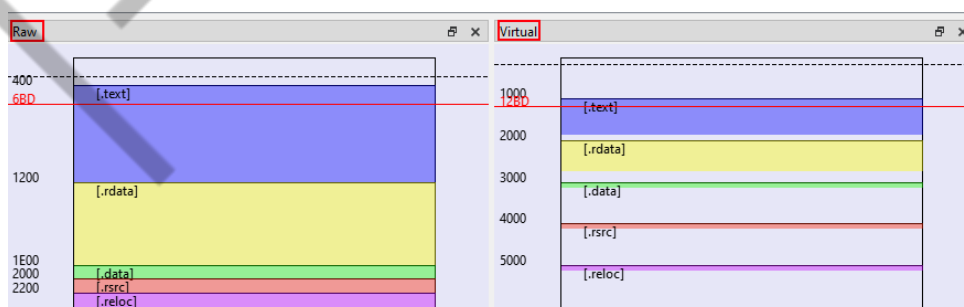


Figura 12 – Exemplo do arquivo PE em disco e em memória
Fonte: Google Imagens (2023)

Como podemos ver na figura “Exemplo do arquivo PE em disco e em memória”, a esquerda representa as seções do binário no disco, já a direita vemos a representação dela ao ser alocada na memória. Mais à frente aprenderemos como realinhar as seções de um binário após realizar seu dump da memória.

4.5 Bound Imports

Antes de começarmos a falar sobre as tabelas de diretórias de importação (*Import Directory Table*) vamos entender o conceito de importação encadeada (*Bound Imports*), esse é um simples conceito, porém de muito importante usando pelo sistema operacional Windows.

A técnica de importação encadeada (*Bound Import*) permite uma ligação antecipada das importações em aberto durante a compilação do programa. Em vez de deixar as referências de importações em aberto e depender do sistema operacional para resolvê-las dinamicamente, o *Bound Import* pré-determina aos endereços das funções ou símbolos de importação. Essa abordagem possui diversas vantagens em relação a otimização e eficiência no tempo necessário para carregar e inicializar o programa.

O uso de importação encadeada ajuda na otimização do programa, dessa forma reduzindo o tempo necessário para o carregador resolver o endereço da função e preencher a IAT (*Import Address Table*). Em casos em que o endereço encadeado não corresponde ao endereço da IAT, então o carregador terá que resolver novamente e reorganizar o endereço da IAT.

Sua estrutura consiste em uma matriz de estruturas `IMAGE_BOUND_IMPORT_DESCRIPTOR` terminando com um valor zero. Abaixo podemos ver essa estrutura:

```
typedef struct _IMAGE_BOUND_IMPORT_DESCRIPTOR {
    DWORD   TimeDateStamp;
    WORD     OffsetModuleName;
    WORD     NumberOfModuleForwarderRefs;
    // Matriz de zero ou mais IMAGE_BOUND_FORWARDER_REF
} IMAGE_BOUND_IMPORT_DESCRIPTOR, * PIMAGE_BOUND_IMPORT_DESCRIPTOR;
```

Código-fonte 5 – Exemplo da estrutura de Bound Import `IMAGE_BOUND_IMPORT_DESCRIPTOR`
Fonte: Elaborada pelo autor (2023)

- **TimeDateStamp (4 Bytes):** A data e horário da DLL importada.
- **OffsetModuleName (2 Bytes):** Um offset contendo o início do nome da DLL importada.

- **NumberOfModuleForwarderRefs (2 Bytes):** O número de estruturas `IMAGE_BOUND_FORWARDER_REF` que seguem imediatamente essa estrutura. `IMAGE_BOUND_FORWARDER_REF` é uma estrutura idêntica a `IMAGE_BOUND_IMPORT_DESCRIPTOR`, a única diferença é que o último membro é reservado.

4.6 Import Direcotory Table (IDT)

Agora iremos entender a importância e como a Tabela de Diretórios de Importação (ou Import Direcotry Table) funciona. Essa tabela é usada para armazenar informações sobre as funções ou símbolos externos de outros módulos ou bibliotecas compartilhadas (DLLs).

Essa tabela é localizada no início da seção (**.idata**), contendo uma lista de estrutura chamada `IMAGE_IMPORT_DESCRIPTOR`, cada descritor de importação representa uma biblioteca externa da qual o arquivo PE realiza o importe das funções. Essa estrutura não possui um tamanho fixo, portanto, o último membro dessa estrutura é zerado, indicando o final da Import Direcotry Table. Abaixo podemos ver a estrutura que define essa tabela:

```
typedef struct _IMAGE_IMPORT_DESCRIPTOR {
    union {
        DWORD Characteristics;
        DWORD OriginalFirstThunk;
    } DUMMYUNIONNAME;
    DWORD TimeDateStamp;
    DWORD ForwarderChain;
    DWORD Name;
    DWORD FirstThunk;
} IMAGE_IMPORT_DESCRIPTOR;
typedef IMAGE_IMPORT_DESCRIPTOR UNALIGNED* PIMAGE_IMPORT_DESCRIPTOR;
```

Código-fonte 6 – Exemplo da estrutura `IMAGE_IMPORT_DESCRIPTOR`

Fonte: Elaborado pelo autor (2023)

- **OriginalFirstThunk (4 Bytes):** Esse membro de uma union armazena o valor do RVA do ILT (Import Lookup Table).
De acordo com a documentação da Microsoft o membro *Characteristics* é usando em **winnt.h**, mas não deve ser descrito nesse campo.

- **TimeDateStamp (4 bytes):** Armazena um carimbo da data (ou timestamp), esse membro é inicializado com o valor 0. Caso uma imagem não seja vinculada (ou encadeada). Depois de ser vinculada a imagem, esse membro recebe a data e horas da DLL.
- **ForwarderChain:** O índice da primeira referência do encaminhador.
- **Name:** O endereço de uma string ASCII que contém o nome da DLL. Este endereço é relativo à base da imagem.
- **FirstThunk:** O RVA da tabela de endereços de importação. O conteúdo desta tabela é idêntico ao conteúdo da tabela de pesquisa de importação até que a imagem seja vinculada.

Existem duas formas principais de se trabalhar com a IDT:

1. Importação de DLL por nomes: Essa é a mais comum usada e permite que as funções sejam importadas com base em seus nomes simbólicos.
2. Importação de DLL por ordem: Já essa é menos comum e trabalha com a base da função em sua posição de ordem na tabela de exportação da biblioteca.

O IDT desempenha um papel fundamental na criação de programas modulares e na reutilização de código. Permitindo com que os desenvolvedores dividam suas aplicações em módulos separados, chamando funções de bibliotecas externas quando necessários. Isso facilita na manutenção e atualização de software.

4.7 Import Lookup Table (ILT)

Quando compilamos um programa, as funções e recursos que serão utilizados para o seu funcionamento são identificados por meio de nomes simbólicos ou identificadores.

A ILT (também conhecida como *Import Name Table* – INT) é uma matriz de números de 32-bit (para PE32) ou uma matriz de número de 64-bit (para PE32+). Essa estrutura mapeia os nomes simbólicos ou identificadores das funções externas de uma biblioteca para os endereços de memória correspondente onde essas funções estão armazenadas. Essa é utilizada em tempo de execução para permitir o acesso correto às funções externas.

Todas DLL que são importadas possuem uma ILT que armazena seu RVA no membro da estrutura **IMAGE_IMPORT_DESCRIPTOR.OriginalFirstThunk**. Sua última entrada é definida como zero (ou NULL) para indicar o final da tabela.

Bit(s)	Tamanho	Campo do Bit	Descrição
31 / 63	1	Ordinal/Nome da Flag	Caso esse bit seja atribuído, o importe é feito de forma ordinal. Caso contrário é feito pelo nome.
15 - 0	16	Número Ordinal	Um número ordinal de 16-bit. Esse campo é utilizado apenas se o campo <i>Ordinal / Nome da flag</i> for atribuído para 1 (ou seja, importado de forma ordinal). Bits 30 - 15 ou 65 - 15 devem ser atribuídos a 0.
30 - 0	31	Hint/Nome da tabela RVA	Um RVA de 31 bits de uma entrada de tabela de dica/nome. Este campo é usado apenas se o campo <i>Ordinal/Name Flag bit</i> for 0 (importar por nome). Para PE32+, os bits 62-31 devem ser zero.

Tabela 2 – As entradas para as importações de uma determinada DLL
Fonte: Elaborado pelo autor (2023)

4.8 Import Address Table (IAT)

A Import Address Table (ou IAT) é uma estrutura de dados que armazena os endereços das funções importadas por um binário de uma DLL. Por exemplo, digamos que queremos utilizar a função `MessageBoxA` para exibir uma mensagem de erro para o usuário. Como podemos ver no código abaixo tudo está certo, fazemos o uso correto da função.

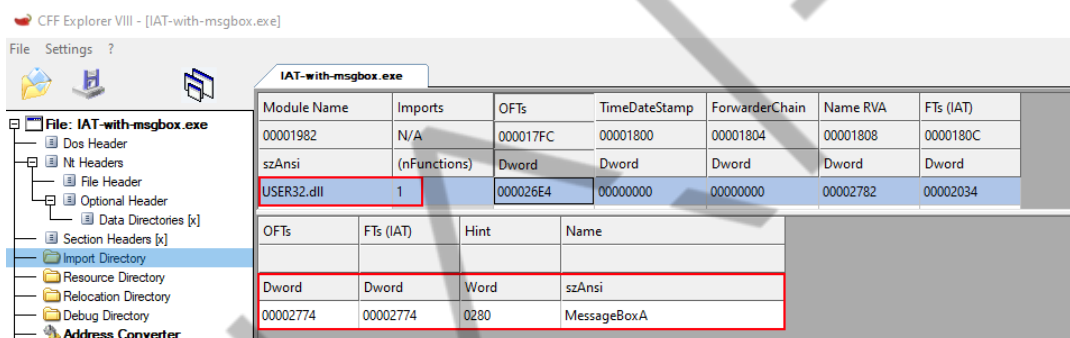
```
#include <Windows.h>

int main()
{
    MessageBoxA(
        NULL,
        "Mensagem de Erro!",
        "!!! Alerta de erro !!!",
        MB_ICONERROR | MB_OK
    );

    return EXIT_SUCCESS;
}
```

Código-fonte 7 – Exemplo de uso da IAT com a função MessageBoxA
Fonte: Elaborado pelo autor (2023)

E após compilarmos o nosso programa, as seguintes informações serão gravadas na IAT, como o nome da função importada pelo programa, como podemos ver na figura “Exemplo da função MessageBox gravada na IAT”.



Module Name	Imports	OFTs	TimeDateStamp	ForwarderChain	Name RVA	FTs (IAT)
00001982	N/A	000017FC	00001800	00001804	00001808	0000180C
szAnsi	(nFunctions)	Dword	Dword	Dword	Dword	Dword
USER32.dll	1	000026E4	00000000	00000000	00002782	00002034

OFTs	FTs (IAT)	Hint	Name
Dword	Dword	Word	szAnsi
00002774	00002774	0280	MessageBoxA

Figura 13 – Exemplo da função MessageBox gravada na IAT
Fonte: Google Imagens (2023)

Essa é sua principal funcionalidade. Porém, alguns arquivos maliciosos fazem o uso de diferentes técnicas para contornar mecanismos de defesa que usam o analisa estática como forma de classificação. A técnica de IAT Hiding permite com que funções sejam chamadas em tempo de execução, possibilitando o ator malicioso esconder as funções que mais chama a atenção.

4.8.1 Iat Hiding

Há duas formas de se fazer de se fazer chamadas dinâmicas para funções, e para isso precisaremos utilizar três funções:

- `GetProcAddress`: Essa função obtém o endereço de uma função exportada.
- `GetModuleHandle` ou `LoadLibrary`: A primeira recebe um handle para um modulo especificado. Já a segunda carrega um modulo no endereço do processo que a chama.

O uso de cada uma depende da situação. Porém, é comumente visto o uso da `LoadLibrary`.

Nesse nosso exemplo utilizaremos a `LoadLibrary`, como podemos ver abaixo:

```
#include <Windows.h>

typedef int (WINAPI* fnMessageBoxA)(
    HWND hWnd,
    LPCSTR lpText,
    LPCSTR lpCaption,
    UINT uType
);

int main()
{
    fnMessageBoxA my_MsgBoxA = GetProcAddress(LoadLibraryA("User32.dll"),
    "MessageBoxA");

    my_MsgBoxA(
        NULL,
        "Mensagem de Erro!",
        "!!! Alerta de erro !!!",
        MB_ICONERROR | MB_OK
    );

    return EXIT_SUCCESS;
}
```

Código-fonte 8 – Exemplo de chamada dinâmica de função (IAT Hiding)
Fonte: Elaborado pelo autor (2023)

Basicamente o conceito é recriar a função e seus tipos, como mostra no começo do código. Em seguida utilizamos o `GetProcAddress` para obter o endereço da `MessageBoxA` da biblioteca do `User32.dll`.

Após compilarmos novamente o código, como podemos observar na Figura 14 abaixo, nem o endereço da função e sua DLL são gravadas na IAT do programa. Porém é gravado as duas funções que utilizamos para chamar a `MessageBoxA`.

CFF Explorer VIII - [IAT-witout-msgbox.exe]

File Settings ?

File: IAT-witout-msgbox.exe

Import Directory

Module Name	Imports	OFIs	TimeDateStamp	ForwarderChain	Name RVA	FTs (IAT)
00001982	N/A	000017FC	00001800	00001804	00001808	0000180C
szAnsi	(nFunctions)	Dword	Dword	Dword	Dword	Dword
KERNEL32.dll	14	0000269C	00000000	00000000	00002782	00002000
VCRUNTIME140.dll	4	000026D8	00000000	00000000	000027E8	0000203C
api-ms-win-crt-run...	19	00002704	00000000	00000000	000029BC	00002068
api-ms-win-crt-mat...	1	000026FC	00000000	00000000	000029DE	00002060
api-ms-win-crt-stdi...	2	00002754	00000000	00000000	000029FE	000020B8
api-ms-win-crt-loc...	1	000026F4	00000000	00000000	00002A1E	00002058
api-ms-win-crt-hea...	1	000026EC	00000000	00000000	00002A40	00002050

OFIs	FTs (IAT)	Hint	Name
Dword	Dword	Word	szAnsi
00002760	00002760	03E1	LoadLibraryA
00002770	00002770	02C6	GetProcAddress
00002B54	00002B54	039D	IsDebuggerPresent
00002B3E	00002B3E	0381	InitializeListHead
00002B24	00002B24	0303	GetSystemTimeAsFileTime
00002B0E	00002B0E	0231	GetCurrentThreadId
00002AF8	00002AF8	022D	GetCurrentProcessId
00002ADE	00002ADE	046D	QueryPerformanceCounter
00002AC2	00002AC2	03A5	IsProcessorFeaturePresent
00002AAE	00002AAE	05B4	TerminateProcess

Figura 14 – Exemplo de IAT Hiding
Fonte: Google Imagens (2023)

Em resumo, a IAT do programa no indica quais funções são utilizadas pelo binário. Mas lembre-se isso não quer dizer que de fato o programa principal faz o uso de todas essas. No nosso exemplo no Código 7, apenas fazemos o uso da `GetProcAddress` e `LoadLibrary`. As outras são incluídas pelo compilador, por diversos motivos, que estão fora do nosso escopo.

4.9 Base Relocation

Quando um arquivo PE é carregado para memória para ser executado, se faz necessário realizar alguns ajustes ou realocação dos endereços de memória. Isso ocorre, por exemplo, quando um executável é carregado em uma localização de memória diferente devido a colisão de endereços ou quando ele é carregado em um local específico solicitando pelo SO.

O objetivo das relocações de base PE é ajustar todos os endereços de memória relativos dentro do executável para que eles apontem corretamente para as posições de memória atualmente utilizadas. Isso é feito adicionando um valor de deslocamento a cada endereço de memória relativo para que ele corresponda à nova posição do executável na memória. Essas relocações são armazenadas em uma tabela de

realocação (*Relocation Table*). Normalmente encontradas na seção (**.reloc**), esse processo é feito pelo *Image Loader*.

A tabela de realocação contém uma lista de entradas, onde cada entrada corresponde a uma relocação de base específica no executável. Cada entrada da tabela de realocação geralmente inclui informações, como o tipo de relocação (por exemplo, uma relocação de 32 bits ou uma relocação de 64 bits) e o local do endereço de memória relativo que precisa ser ajustado.

5 EXECUTANDO BINÁRIO WINDOWS EM SISTEMAS GNU/LINUX

Sim, é isso mesmo que você leu! É possível executar binários do Windows em sistemas GNU/Linux. Você deve estar se perguntando como isso é possível, se o sistema Linux utiliza o formato ELF (*Executable and Linkable Format*) para execução de seus binários. Tudo isso é possível graças ao WINE.

WINE é um software de código aberto distribuído sobre a licença (LGPL-2.1-or-later) que se originou por volta de 1993. A ideia principal para esse projeto era o desenvolvimento de um software capaz de executar binário Windows em sistemas GNU/Linux. A sigla WINE significa (*Wine Is Not an Emulator*), ou seja, esse não é um emulador e nem uma máquina virtual.

- **Hypervisor (Virtual Machine Monitor):** Um Hypervisor é uma camada de software ou firmware que possibilita a criação e o gerenciamento de múltiplas Virtual Machines (VMs) em um computador físico (também conhecido como Máquina Host). Esse tipo de software promove uma camada de abstração que nos permite que vários sistemas operacionais sejam executados simultaneamente no mesmo hardware. A função principal do Hypervisor é gerenciar a alocação de recursos, como CPU (*Central Processing Unit*), memória primária, memória secundária, e conectividade de rede para a máquina Guest. Existem diversos software para realizar essa função, alguns exemplos são:
 - VMware ESXI.
 - Microsoft Hyper-V.

- Oracle VirtualBox.
- KVM (Kernel-based Virtual Machine).
- **Emulador:** Já um emulador pode ser uma aplicação ou um sistema que reproduz o comportamento de diferentes sistemas computacionais ou plataformas. Permitindo a execução de diferentes softwares desenvolvidos para uma plataforma específica. Os emuladores realizam uma “mímica” da arquitetura e das instruções de uma determinada arquitetura de sistema possibilitando a execução de tais software. Um simples exemplo seria, a emulação de jogos de vídeo games desenvolvidos para consoles com o Super Nintendo. Emuladores também oferecem uma simulação completa do sistema como processador, memória, e dispositivos I/O, entre outros componentes.

Agora que entendemos a diferença entre VM e Emuladores, podemos contextualizar o WINE sem preocupações e dúvidas. De forma resumida **o WINE é uma camada que performa uma tradução de aplicações Windows para o Sistema GNU/Linux**. Mas vamos entender com mais detalhes. Para isso, vamos pegar como exemplo a figura “Composição de um sistema operacional I” para podermos visualizar e contextualizar da melhor forma:

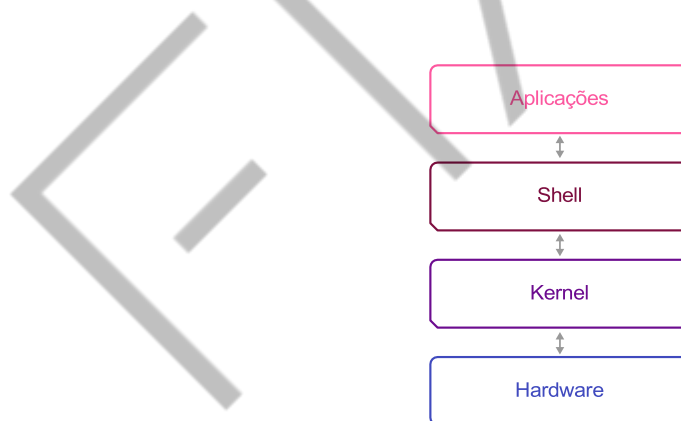


Figura 15 – Composição de um sistema operacional I
Fonte: Google Imagens (2023)

Acima, é representada de forma muito básica a composição de um sistema operacional, independentemente de ser Windows, GNU/Linux, ou Mac-OS. Todos SO seguirão basicamente essa ideia. Começando de baixo para cima, o Kernel é responsável por se comunicar entre a Shell e os periféricos do hardware do computador. O Shell (ou Casca) é uma interface (tanto CLI como GUI) que permite a interação com o Kernel. Já as aplicações são programas que realizam chamadas de

sistemas para se comunicar com o Kernel e por exemplo um navegador se comunicando com a internet e renderizar uma imagem na tela do usuário.

Então de uma forma bem simples, vamos imaginar que o sistema Windows fala português, e o Linux inglês. Para uma aplicação Windows se comunicar com o Linux é preciso realizar uma tradução e é aí que entra o WINE.

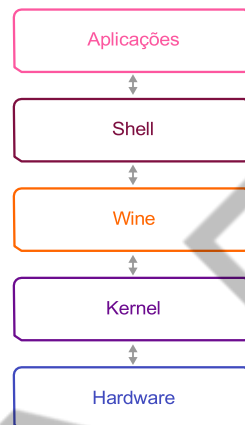


Figura 16 – Composição de um sistema operacional II
Fonte: Google Imagens (2023)

Como podemos ver na figura “Composição de um sistema operacional II”, o WINE fica entre o Kernel e o Shell dessa forma realizando a tradução das funções do Windows para as funções do GNU/Linux.

Para isso acontecer o WINE não cria um ambiente completo do Windows para executar os programas. Em vez disso, ele fornece uma implementação das bibliotecas (DLLs) do Windows necessárias para que o programa seja executado diretamente no ambiente Linux. Dessa forma performando a tradução das chamadas de sistemas Windows para chamadas de sistemas nativas do Linux.

5.1 Vantagens do WINE

As vantagens que o WINE nos possibilita são diversas, nos possibilitando executar nossos jogos favoritos em um ambiente Linux, permitindo economizar muito mais memória. O uso de softwares desenvolvidos especificamente para o sistema Windows como o Office 365.

Já como profissional de segurança, digamos que você é um analista de malware. Existem diferentes técnicas que um software malicioso pode performar para

validar se ele está em um ambiente virtual, buscando por processo, por arquivos, e até instruções de CPU. Não que isso também não aconteça com o WINE, alguns malware também realiza essa validação.

5.2 Instalando o WINE

Agora vamos aprender a instalar o como utilizar o WINE. Nesse tutorial irei utilizar um Ubuntu 20.04 LTS. Então vamos lá. Nesse tutorial estamos utilizando a versão (8.0.1) mais recente na data em que estamos escrevendo esse material.

A seguir estão os passos necessários para realizar a instalação correta do WINE.

Precisamos validar a versão no sistema Linux, no nosso caso estamos utilizando a versão x64. Para validar execute o comando `uname -r` esse nos retornará a arquitetura do SO.

Em seguida precisamos criam um diretório que irá armazenar nossa chave e em seguida precisamos fazer o download da chave. Para isso execute os comandos:

1. `sudo mkdir -pm755 /etc/apt/keyrings` – Cria um novo diretório com permissão de leitura, escrita e execução.
2. `sudo wget -O /etc/apt/keyrings/winehq-archive.key https://dl.winehq.org/wine-builds/winehq.key` – Faz o download da chave para validar a autenticação dos pacotes do WineHQ.
3. Caso no seu sistema não possua o `wget`, execute o seguinte comando: `sudo apt install wget`.
4. `sudo apt update` – atualiza os repositórios de pacotes.
5. `sudo apt install --install-recommends winehq-stable` – Realiza a instalação da última versão estável do WINE.

Caso tudo tenha ocorrido bem execute o comando `wine --version` para validar se está tudo certo mesmo.

Para executar um binário Windows no WINE basta executá-lo seguindo a seguinte sintaxe: `wine <caminho do programa>.exe`.

Com isso concluímos nossa instalação do WINE.

CONCLUSÃO

Este capítulo sobre binários Windows proporcionou uma visão abrangente dos diferentes tipos de arquivos binários e do formato PE (Portable Executable) utilizados no sistema operacional Windows. Exploramos os elementos essenciais de um arquivo PE, incluindo o DOS Header, o DOS Stub, o Rich Header e a estrutura do NT Header. Além disso, discutimos as diversas seções e diretórios de dados presentes em um arquivo PE, como Bound Imports, Import Directory Table (IDT), Import Lookup Table (ILT) e Base Relocation.

Também abordamos a execução de binários Windows em sistemas GNU/Linux, destacando as vantagens oferecidas pelo WINE, uma camada de compatibilidade que permite a execução de aplicativos do Windows em sistemas operacionais baseados em UNIX. Fornecemos instruções sobre a instalação do WINE, o que abre um mundo de possibilidades para os usuários de GNU/Linux ao permitir que eles utilizem uma ampla gama de softwares desenvolvidos originalmente para o Windows.

Ao compreender a estrutura e o funcionamento dos binários Windows, assim como as opções disponíveis para executá-los em outros sistemas operacionais, os leitores agora têm um conhecimento sólido para explorar e interagir com aplicativos e bibliotecas do Windows de forma mais eficiente. O entendimento desses conceitos é fundamental para aqueles que desejam desenvolver, analisar ou depurar softwares Windows, bem como para aqueles que buscam compatibilidade e interoperabilidade entre diferentes plataformas de sistemas operacionais.

REFERÊNCIAS

DUBYK, Maksim. Leveraging the PE Rich Header for Static Malware Detection and Linking. **GIAC Certifications**, 2019. Disponível em: <<https://www.giac.org/paper/grem/6321/leveraging-pe-rich-header-static-malware-detection-linking/169729>>. Acesso em: 20 jun. 2023.

EMEND